

MODULE 6

Resilient and High-Performance Microservices

Designing distributed services that stay responsive, observable and recoverable.

Designing for
failure

Timeouts and
retries

Circuit breakers

Bulkheads

Observability

SLAs, SLOs and
SLIs

Continuous
testing

LEARNING OBJECTIVES

Failure handling, observability, reliability targets and resilience testing.

BY THE END OF THIS MODULE, YOU CAN

1. Explain failure in distributed systems
2. Configure appropriate timeouts
3. Apply safe retry strategies
4. Explain exponential backoff and jitter
5. Implement circuit-breaker concepts
6. Apply bulkhead isolation
7. Design graceful degradation
8. Use logs, metrics and traces
9. Distinguish SLI, SLO and SLA
10. Continuously test resilience

FOUR SECTIONS

1 Failure handling

Timeout

Retry

Circuit
Breaker

Bulkhead

Fallback

2 Observability

Logs

Metrics

Traces

3 Reliability targets

SLI

SLO

SLA

4 Validation

Test

Measure

Fail

Recover

Improve

KEY TAKEAWAY Learn the individual patterns — and how they work together in production.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 **Timeout**

The longest a call may wait before it gives up.

2 **Retry with backoff**

Trying a failed call again after growing, slightly randomized delays.

3 **Circuit breaker**

Stops calling a failing dependency for a while, so calls fail fast.

4 **Bulkhead**

Limits what one dependency can use, so it can't exhaust the whole service.

5 **Fallback**

A safe response when a dependency fails — never an invented success.

6 **Correlation ID**

One identifier carried through every service and log line for a single request.

7 **Liveness and readiness**

Liveness means restart me; readiness means don't send me traffic yet.

8 **Service level objective**

The reliability target a team commits to, such as 99.9% successful requests.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

API	Application Programming Interface	GC	Garbage Collection	SLA	Service Level Agreement
CI	Continuous Integration	HTTP	HyperText Transfer Protocol	SLI	Service Level Indicator
CI/CD	Continuous Integration / Continuous Delivery	ID	Identifier	SLO	Service Level Objective
CPU	Central Processing Unit	JSON	JavaScript Object Notation	UUID	Universally Unique Identifier
DB	Database	JVM	Java Virtual Machine		
DNS	Domain Name System	MDC	Mapped Diagnostic Context		

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 5 TO MODULE 6

Services own their data and talk over the network — now keep them standing when calls fail.

MODULE 5 GAVE US

- ✓ Service-owned data and local transactions
- ✓ Sagas and compensation across services
- ✓ Idempotent consumers and the outbox
- ✓ Redis caching with TTLs
- ✓ Many services calling each other

NOW Keep them responsive when those calls get slow or fail.



OUR PATH THROUGH MODULE 6

- 1 Why failure is normal — and spreads
- 2 Timeouts, retries and idempotency
- 3 Circuit breakers, bulkheads, fallbacks
- 4 Logs, metrics and traces
- 5 Health checks, SLOs and capacity
- 6 Test failure before it happens
- 7 The complete resilient architecture

Part 1

Part 1

Part 1

Part 2

Part 3

Part 4

Part 4

KEY TAKEAWAY Module 5 split the data; Module 6 keeps the whole system responsive when parts of it fail.

PART 1: FAILURE HANDLING

Contain failures so one slow dependency can't take everything down.



SO FAR

- ✓ Services own their data and commit locally
- ✓ Sagas and events coordinate across services
- ✓ Every cross-service step is a network call

NEXT QUESTION

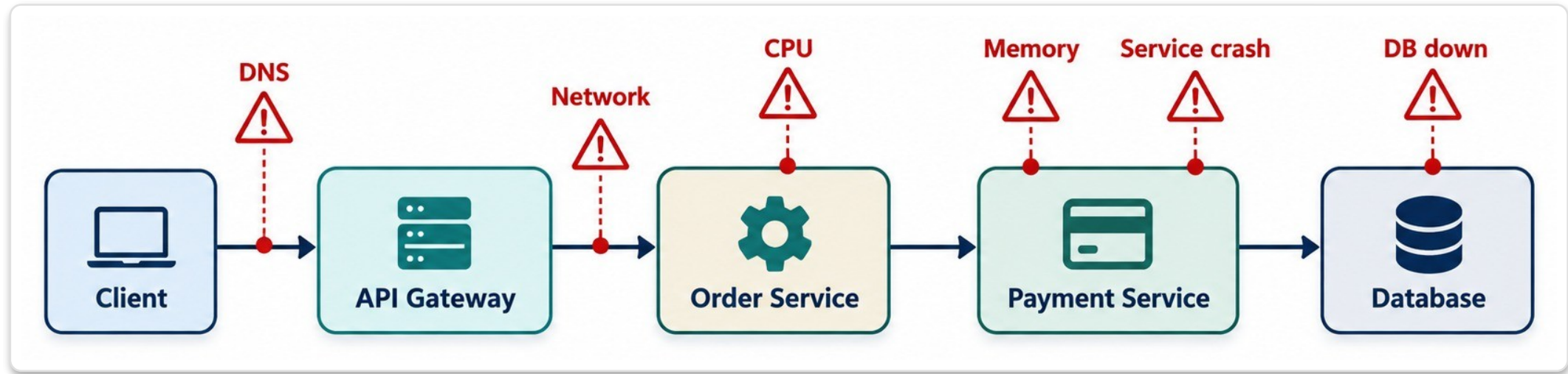
A dependency is slow or down — how do we stop it taking our service down too?

IN THIS PART

- 1 Why failure is normal — and how it cascades
- 2 Designing for failure
- 3 Timeouts and timeout budgets
- 4 Retries, backoff, jitter and idempotency
- 5 Circuit breakers and bulkheads — in code
- 6 Combining patterns and degrading gracefully

WHY FAILURE IS NORMAL IN DISTRIBUTED SYSTEMS

Networks, dependencies, pods, databases, APIs and infrastructure fail independently.



A "simple" call

```
paymentService.processPayment();
```

...REALLY DOES ALL OF THIS

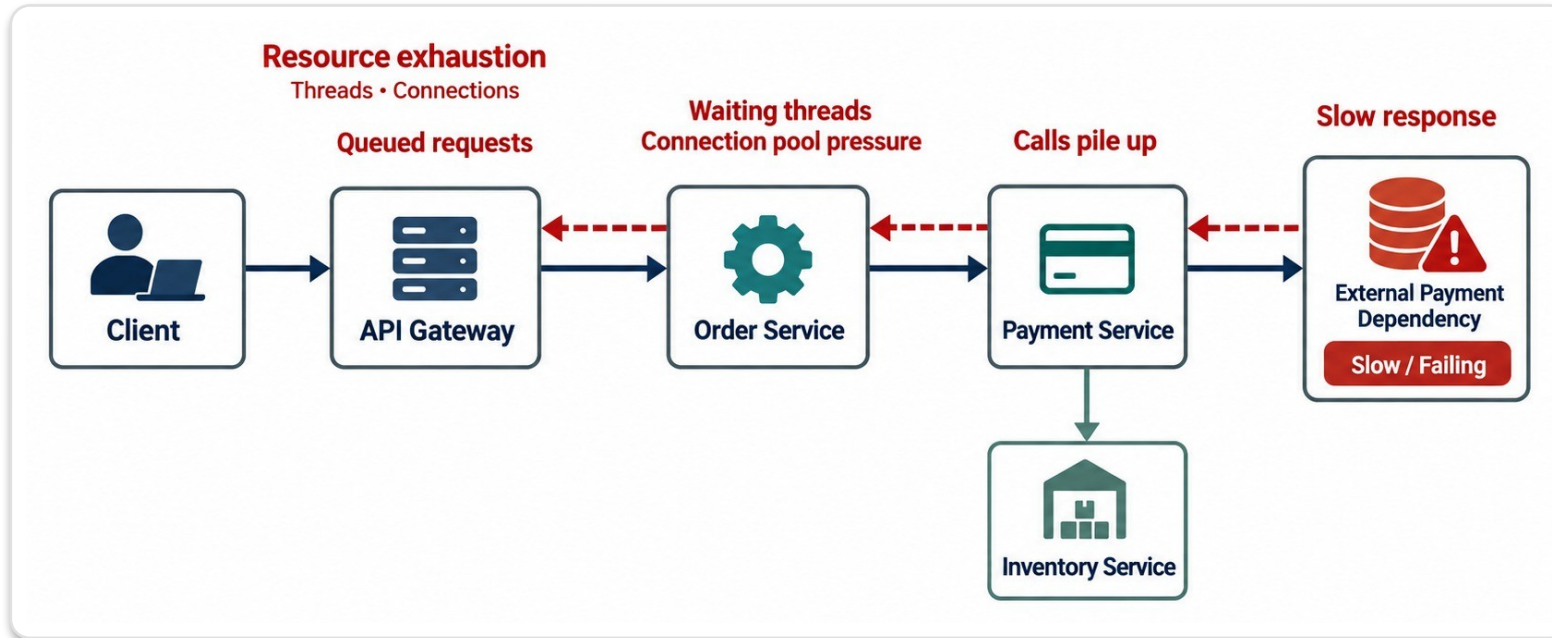
- ✓ Resolve DNS, open a connection
- ✓ Serialize and send
- ✓ Remote work and its dependencies
- ✓ Response back, deserialize

REMEMBER Every remote dependency can fail independently.

KEY TAKEAWAY Remote calls are neither instantaneous nor guaranteed to succeed.

FAILURE PROPAGATION IN MICROSERVICES

One slow or failing service spreading latency and failure to its callers.



Normally → suddenly

Payment answers in 200 ms — until its provider makes it 30 seconds.

USERS EXPERIENCE

- ! Slow responses
- ! Timeouts
- ! 503 errors

CASCADING FAILURE Order Service was healthy — it just kept waiting.

KEY TAKEAWAY One slow dependency can drag down every service that waits on it.

DESIGNING FOR FAILURE

Fail fast, isolate failures, degrade gracefully, recover and stay observable.



THE RESILIENCE MINDSET

- ✓ Prevent what you can
- ✓ Contain what you can't prevent
- ✓ Recover from what occurs
- ✓ Learn from what happened

BUILDING BLOCKS

Timeouts

Retries

Circuit breakers

Bulkheads

Rate limits

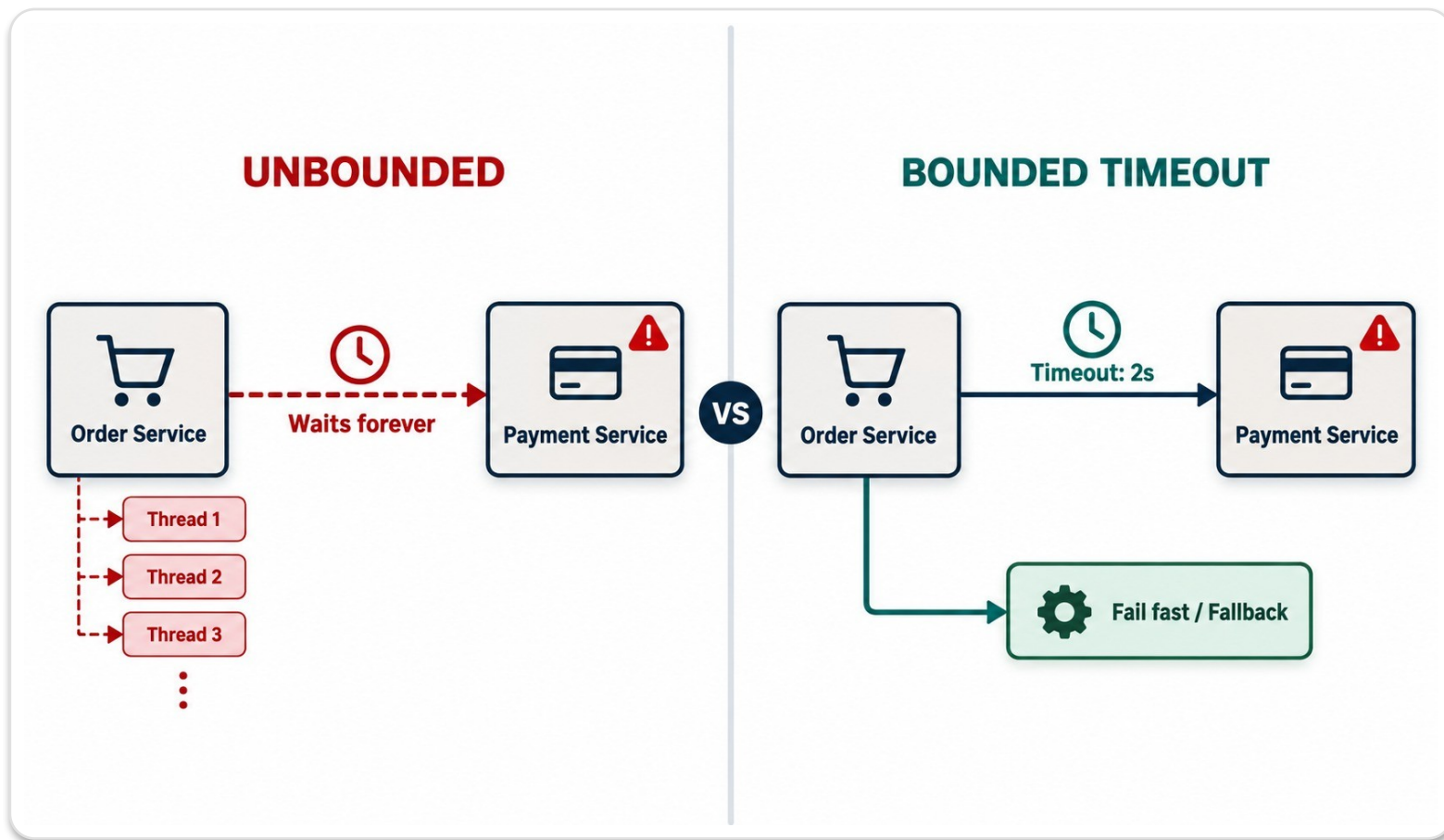
Fallbacks

MINDSET Assume failure will happen, then limit its impact.

KEY TAKEAWAY Assume failure will happen — then deliberately limit its impact.

TIMEOUTS — STOP WAITING FOREVER

Every remote call needs a bounded wait time.



AFTER A TIMEOUT, THE CALLER DECIDES

- ✓ Return an error
- ✓ Retry — if it's safe
- ✓ Use a fallback
- ✓ Start async recovery

EVERY REMOTE CALL NEEDS ONE

HTTP calls

DB queries

Broker operations

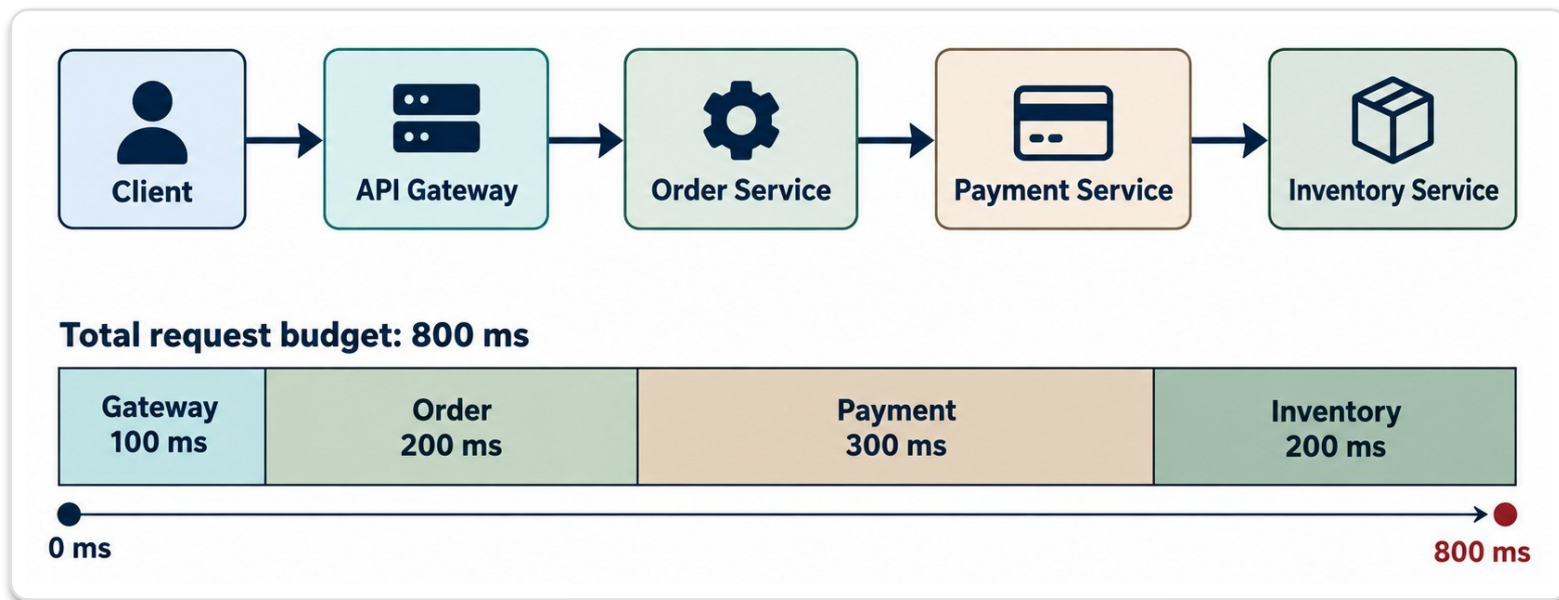
Cache calls

RULE Timeouts turn unbounded waiting into controlled failure.

KEY TAKEAWAY Timeouts convert unbounded waiting into controlled failure.

CHOOSING APPROPRIATE TIMEOUT VALUES

Connection vs. read timeouts, downstream latency, SLAs and user expectations.



Connection timeout

How long to wait to establish a connection.

Read timeout

How long to wait for the response once connected.

THE TRADE-OFF

- ! Too long: failures detected slowly
- ! Too short: healthy slow calls cut off

BUDGET Downstream timeouts must fit inside the user's target.

KEY TAKEAWAY A timeout comes from latency budgets and real behavior — not a number copied from elsewhere.

TIMEOUTS IN SPRING BOOT

Socket timeouts on the HTTP client, plus a time limit on the whole operation.

application.yml

```
# Spring Boot 3.4+: auto-configured RestClient.Builder
spring:
  http:
    client:
      connect-timeout: 2s
      read-timeout: 3s

resilience4j:
  timelimiter:
    instances:
      fraudCheck:
        timeout-duration: 800ms
```

Socket timeouts

Connect 2 s, read 3 s — Lab 3's values for the Account call.

Time limit

Bounds the whole operation, including any queueing.

Fallback

PENDING_REVIEW — never an automatic approval.

✓ FraudClient.java

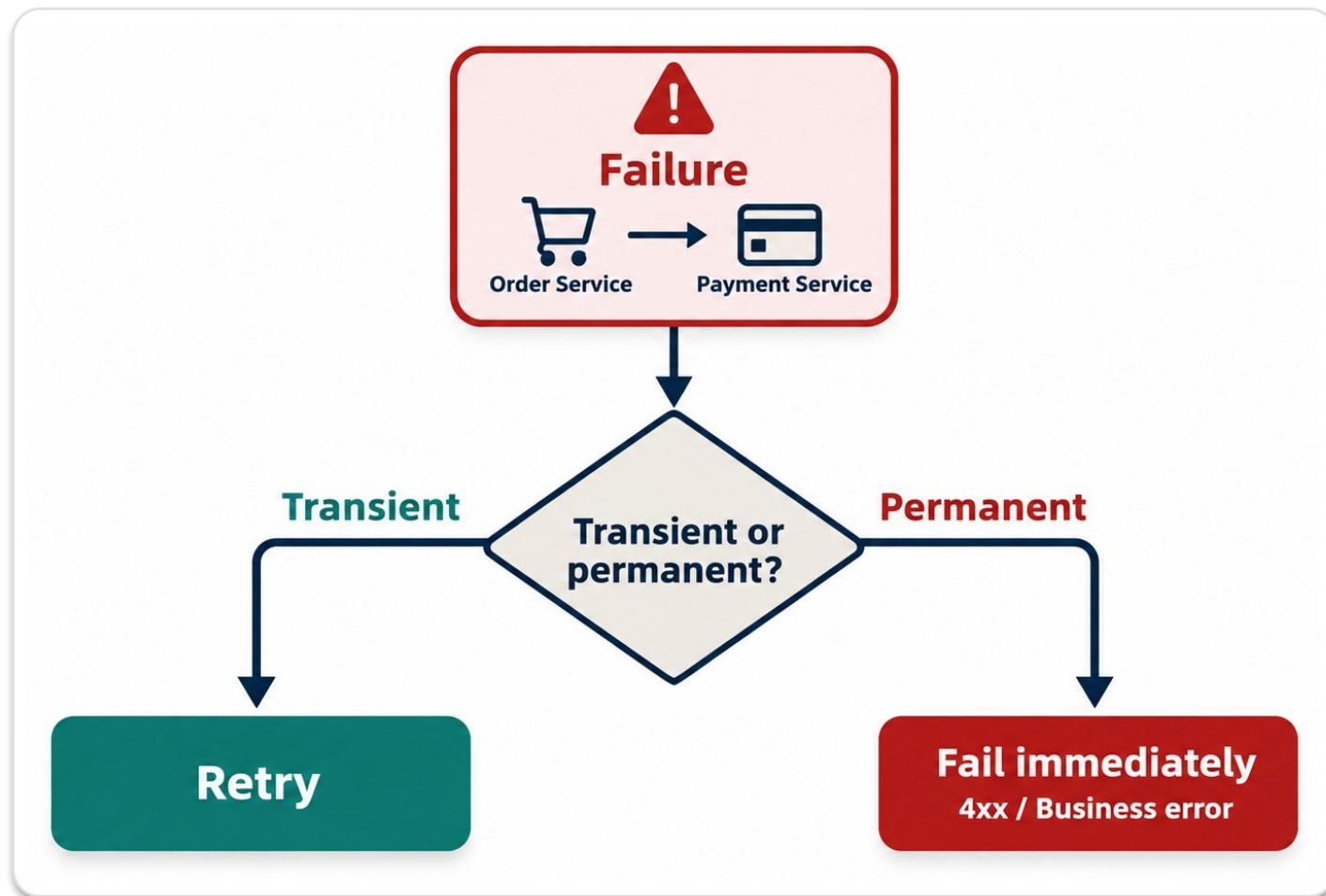
```
@TimeLimiter(name = "fraudCheck", fallbackMethod = "reviewLater")
public CompletableFuture<FraudDecision> check(Payment p) {
    return CompletableFuture.supplyAsync(() -> fraud.check(p));
}

private CompletableFuture<FraudDecision> reviewLater(
    Payment p, TimeoutException ex) {
    return CompletableFuture.completedFuture(
        FraudDecision.pendingReview(p.id()));
}
```

KEY TAKEAWAY Set client timeouts on every call, and bound slow operations with a time limit and a safe fallback.

RETRIES — WHEN THEY HELP AND WHEN THEY HURT

Transient versus permanent failures — retry only the right operations.



RETRY MAY HELP

- ✓ 503 Service Unavailable
- ✓ Connection reset
- ✓ 429 with Retry-After

RETRY WON'T FIX

- ✗ 400 Bad Request
- ✗ 401 or 403
- ✗ Business-rule rejection

Retries add traffic

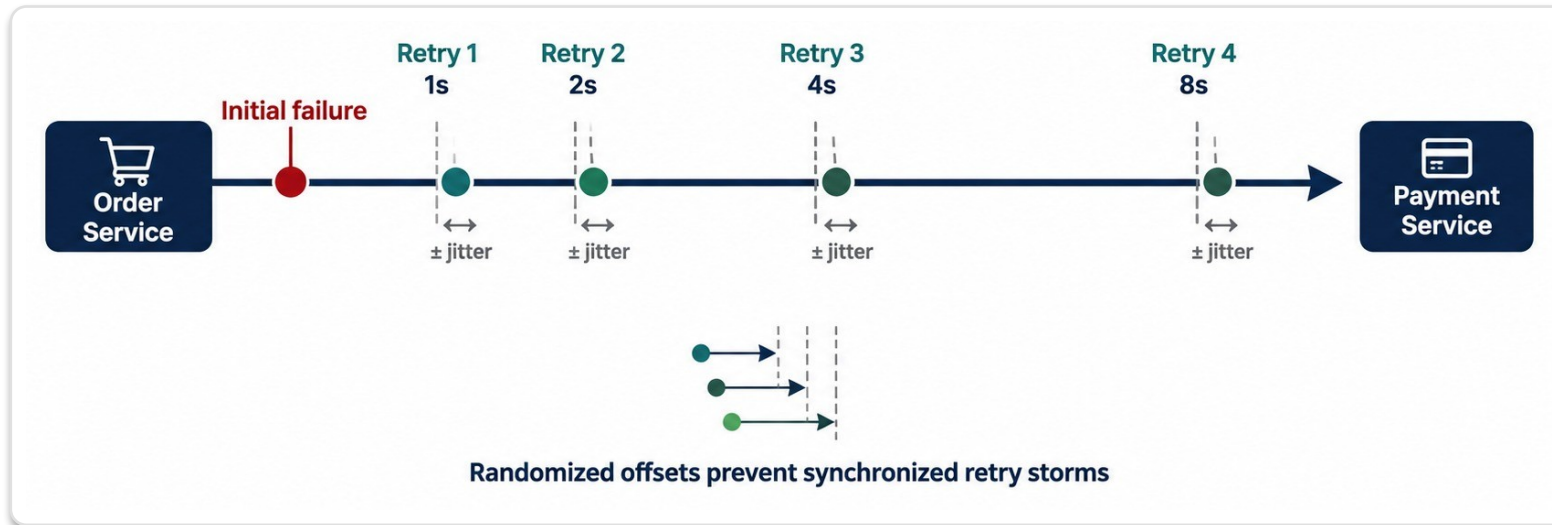
10,000 failures × 3 immediate retries = 30,000 extra calls on a struggling service.

RULE Retry selectively, not automatically.

KEY TAKEAWAY Retries help with transient faults — and hurt when used blindly.

RETRY WITH BACKOFF AND JITTER

Exponential backoff — and avoiding synchronized retry storms.



Without jitter

Thousands of clients retry at exactly 1 second — a synchronized storm.

Always cap it

Limit the attempts and the total wait, inside the latency budget.

FORMULA delay = base $\times 2^n$, plus a random offset.

KEY TAKEAWAY Backoff slows retries down; jitter stops everyone retrying at the same moment.

RETRY WITH BACKOFF IN RESILIENCE4J

Retry only transient failures on safe reads — with backoff, jitter and a fallback.

application.yml

```
resilience4j:
  retry:
    instances:
      exchangeRates:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        enable-randomized-wait: true
        randomized-wait-factor: 0.5
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
        ignore-exceptions:
          - org.springframework.web.client.HttpClientErrorException
```

✓ ExchangeRateService.java

```
@Retry(name = "exchangeRates", fallbackMethod = "lastKnownRate")
public BigDecimal usdToEur() {
    return rateClient.getRate("USD", "EUR");    // a safe read
}

private BigDecimal lastKnownRate(Throwable ex) {
    return rateCache.lastKnown("USD", "EUR");
}
```

Backoff + jitter

500 ms, then about 1 s — each randomized by $\pm 50\%$.

Only transient errors

Timeouts and resets retry; 4xx client errors don't.

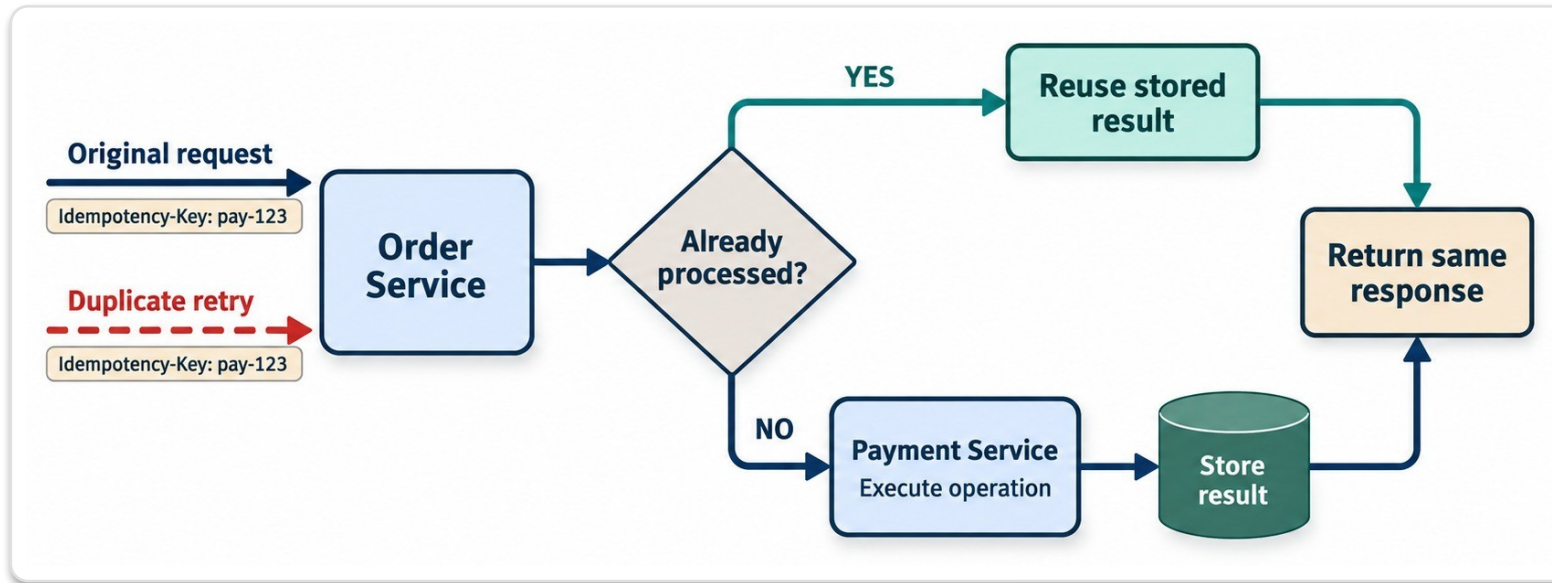
Reads only

Writes need an idempotency key before any retry.

KEY TAKEAWAY Configure retries per dependency: few attempts, growing randomized waits, and only for errors that can recover.

IDEMPOTENCY AND SAFE RETRIES

Retrying GET vs. payment and order operations — idempotency keys.



The client sends a key

POST /payments
Idempotency-Key: pay-123

Server side

Store the key and the result in the same transaction; a repeat returns the stored response.

THINK BUSINESS EFFECT A timeout doesn't tell you whether the charge happened.

KEY TAKEAWAY Retry only when repeating the operation can't create a second business effect.

EXERCISE: RETRY OR NOT?

Decide how each failed call should be handled.

PRACTICE EXERCISE

10 MIN

SCENARIO

Six calls just failed:

- 1 GET exchange rate → 503
- 2 POST transfer → timeout, no key
- 3 POST transfer → timeout, with key
- 4 PUT profile → 400 validation
- 5 GET balance → connection reset
- 6 POST payment → 429, Retry-After: 2

YOUR TASKS

- 1 Decide: retry or fail fast
- 2 For retries, choose attempts and backoff
- 3 Say which case needs an idempotency key — and why

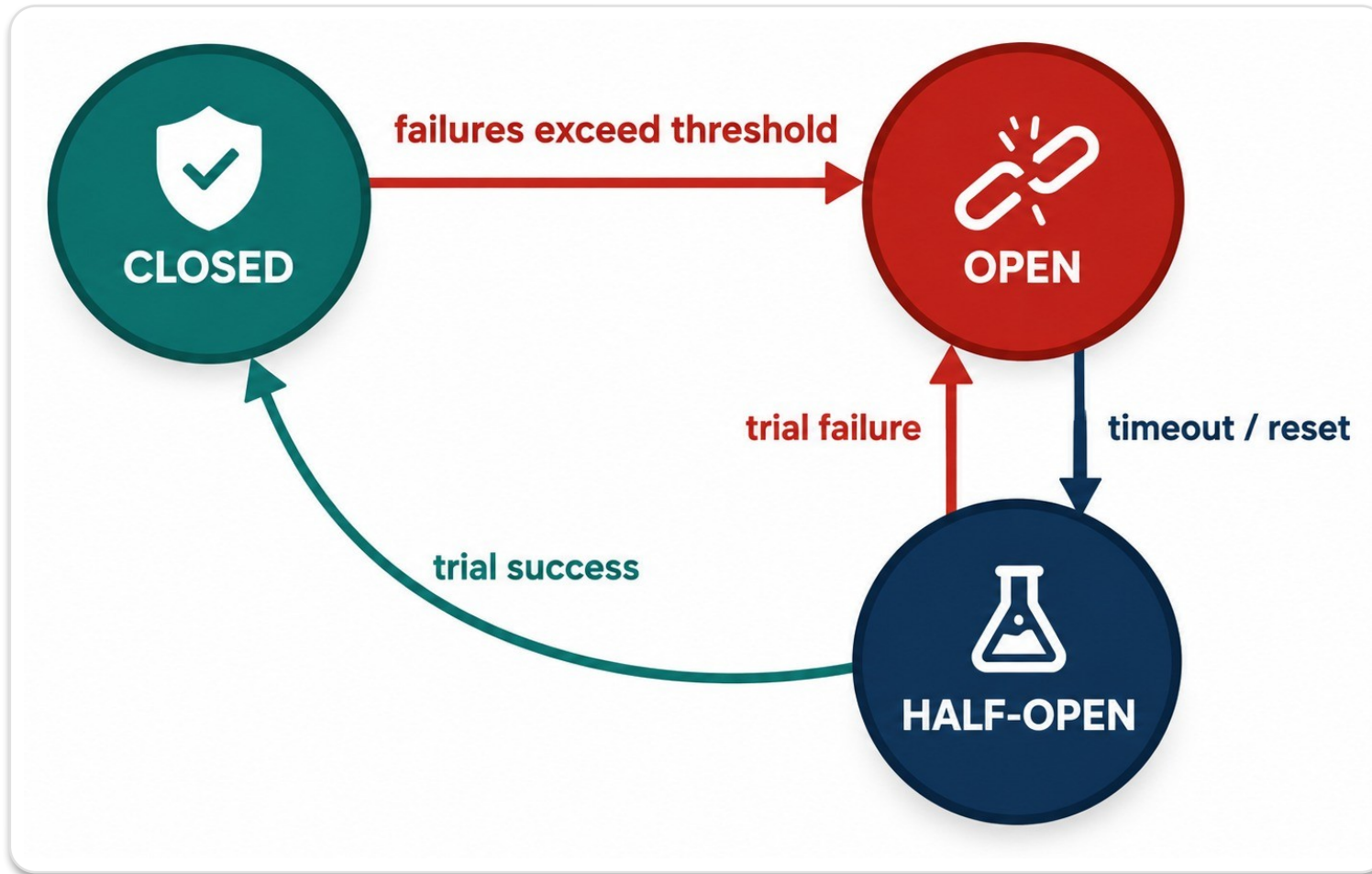
EXPECTED OUTCOME

- ✓ 1, 5: retry with backoff and jitter
- ✓ 2: don't retry — check the transfer's status first
- ✓ 3: retry safely with the same key
- ✓ 4: fail fast — the request must change
- ✓ 6: wait 2 s, then retry once

KEY TAKEAWAY Retry transient failures on safe or keyed operations — and fail fast on everything else.

CIRCUIT BREAKER PATTERN

Closed → Open → Half-Open → Closed.



Closed

Calls flow; failures are counted over a sliding window.

Open

Calls fail fast for the configured wait duration.

Half-open

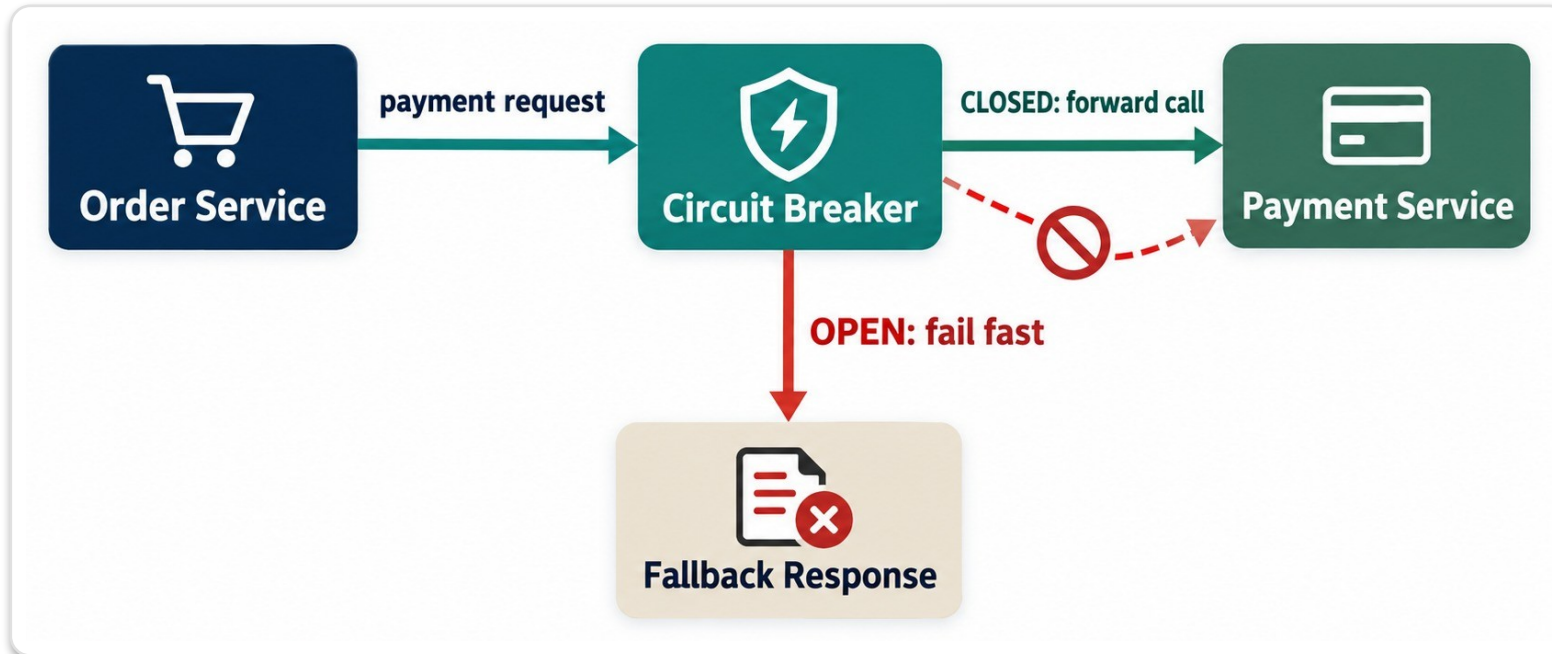
A few trial calls decide: close again or reopen.

LIKE A FUSE BOX It stops continued failure from spreading.

KEY TAKEAWAY Fail fast instead of repeatedly calling a dependency that is already failing.

CIRCUIT BREAKER IN A MICROSERVICE CALL

Order Service → Payment Service with a failure threshold and fallback.



It protects the caller

It doesn't repair Payment Service — it keeps Order responsive.

SAFE FALLBACKS

- ✓ PAYMENT_PENDING
- ✓ A clear 503 with a retry hint

NEVER

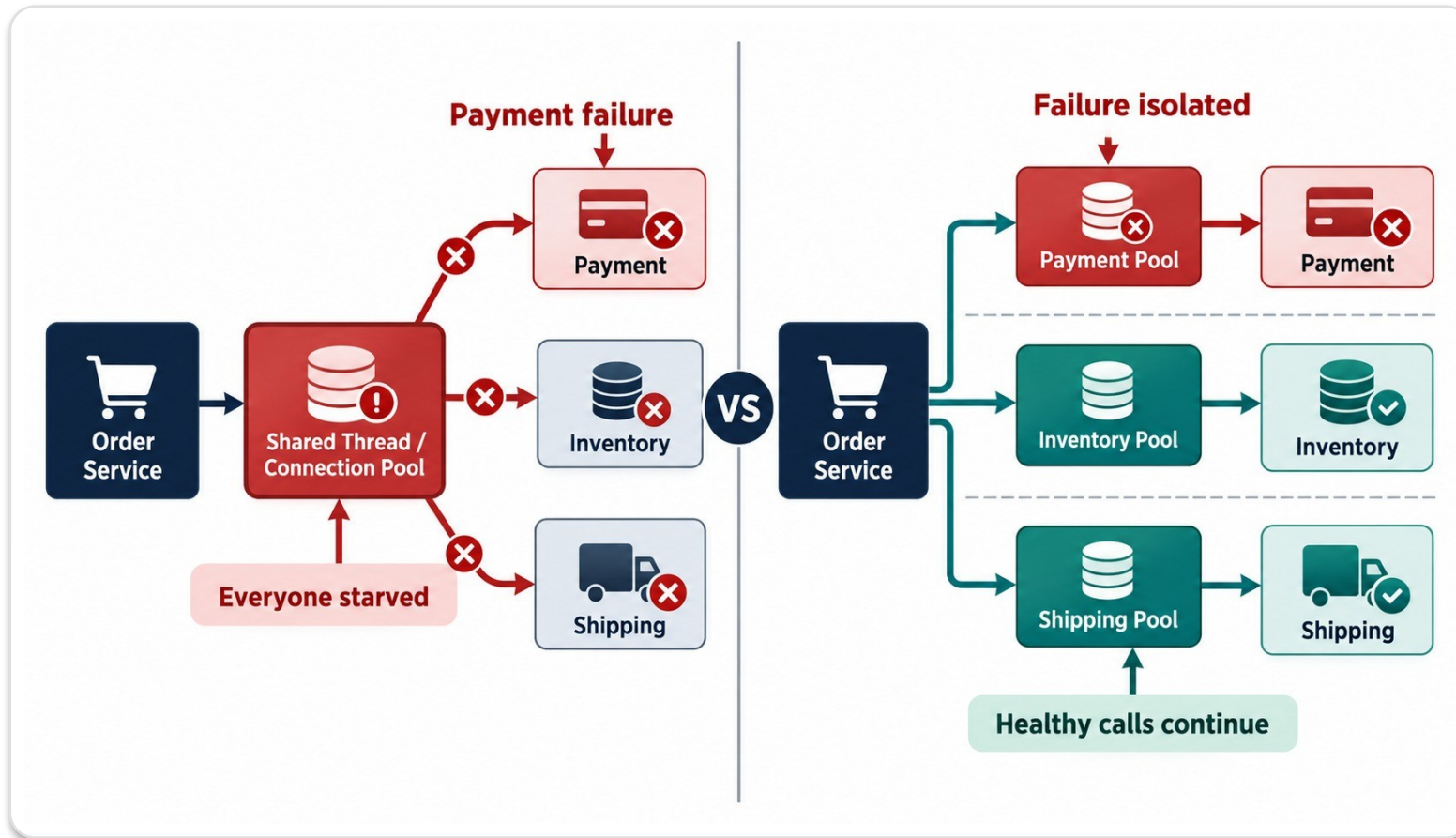
- ✗ "Payment successful"

CRITERIA Failure rate and slow-call rate over a sliding window.

KEY TAKEAWAY When Payment is unhealthy, the breaker keeps Order responsive with a controlled response.

BULKHEAD PATTERN

Isolate resources so one failing dependency can't exhaust everything.



BULKHEADS CAN ISOLATE

Thread pools

Connection pools

Concurrency limits

Queues

Semaphore or thread pool

A semaphore caps concurrent calls cheaply; a thread pool fully separates blocking work.

Rate limiting

A related guard: cap requests per period and answer 429.

LIKE A SHIP One flooded compartment doesn't sink the vessel.

KEY TAKEAWAY A failing dependency should use only its allocated capacity — not the whole service.

CIRCUIT BREAKER AND BULKHEAD WITH RESILIENCE4J

Lab 3's breaker on the Account call — and a safe 503 when it opens.

application.yml

```
resilience4j:
  circuitbreaker:
    instances:
      accountService:
        sliding-window-size: 10
        minimum-number-of-calls: 4
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
        ignore-exceptions:
          - com.example.AccountConflictException # 409
  bulkhead:
    instances:
      accountService:
        max-concurrent-calls: 20
```

Lab 3 values

At least 4 calls, 50% failing, open for 10 s — a 409 never trips it.

Fallback

503 ACCOUNT_SERVICE_UNAVAILABLE — never a fake ACTIVE account.

Bulkhead

Shown for completeness; the labs keep it conceptual.

✓ AccountClient.java

```
@CircuitBreaker(name = "accountService",
                 fallbackMethod = "unavailable")
@Bulkhead(name = "accountService") // concept only in the labs
public Account requireActiveAccount(String accountId) {
    return accountApi.get(accountId);
}

private Account unavailable(String id, CallNotPermittedException e) {
    throw new AccountServiceUnavailableException(); // → 503
}

private Account unavailable(String id, ResourceAccessException e) {
    throw new AccountServiceUnavailableException(); // → 503
}
```

KEY TAKEAWAY Open the breaker on real dependency failures, ignore business conflicts, and fail with a clear 503.

EXERCISE 3.1 — CONFIGURE TIMEOUT AND CIRCUIT BREAKER

Day 3 checkpoint A: Resilience4j on the Account dependency.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-03/exercises/exercise-3.1-timeout-circuit-breaker.md

SCENARIO

Lab 3 hardens the banking Transaction Service. Before accepting money movement, it calls Account Service through `AccountClient.requireActiveAccount`.

Scope: Account, Transaction and Risk Assessment only.

YOUR TASKS

- 1 Find the RestClient timeouts (connect 2 s, read 3 s) in the Lab 3 Transaction starter
- 2 Circuit breaker: 4 failed calls at 50% open it; wait 10 s; later calls fail fast. A 409 does not trip it
- 3 Safe fallback: `POST /api/v1/transactions` returns 503
`ACCOUNT_SERVICE_UNAVAILABLE` — no Kafka event
- 4 Bulkhead and rate limiting stay conceptual — no extra libraries

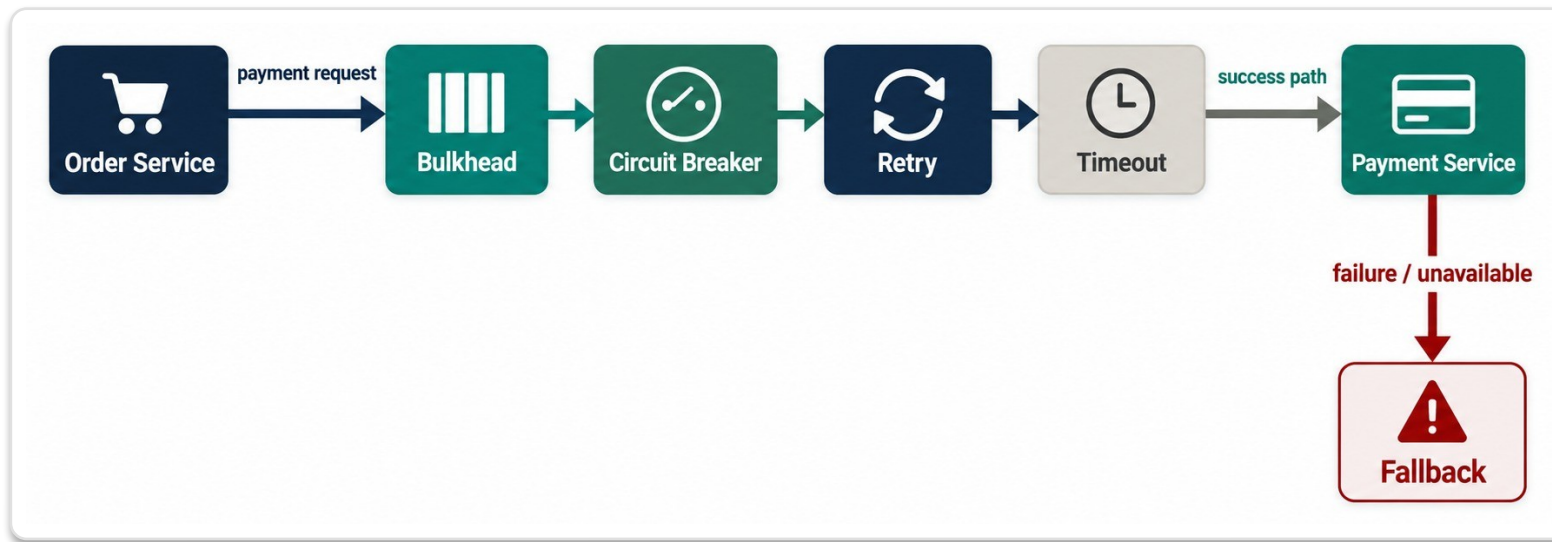
EXPECTED OUTCOME

- ✓ Timeouts located in the starter
- ✓ Breaker settings you can explain
- ✓ 503 fallback with no Kafka event
- ✓ Never a fake ACTIVE account

KEY TAKEAWAY The fallback rule: never invent a successful financial outcome.

COMBINING RESILIENCE PATTERNS

Timeout + retry + circuit breaker + bulkhead + fallback.



EACH LAYER'S JOB

- ✓ Bulkhead: isolate resources
- ✓ Circuit breaker: fail fast
- ✓ Retry: handle transient faults
- ✓ Timeout: bound each attempt
- ✓ Fallback: stay useful

Composition matters

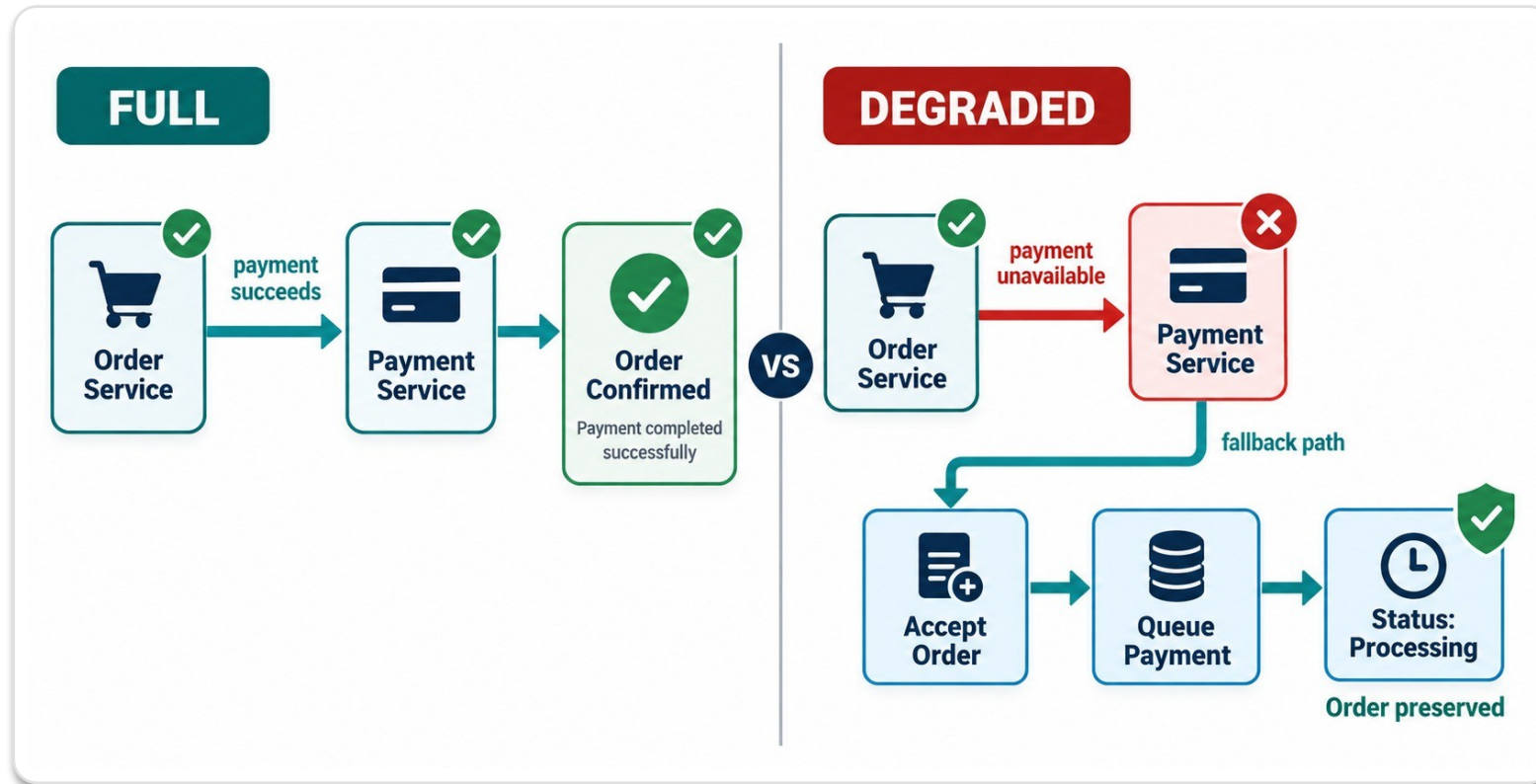
3 retries × a 2 s timeout = 6 s — far beyond a 2 s budget.

ORDER Libraries differ — Resilience4j applies Retry outermost by default.

KEY TAKEAWAY Design the whole resilience policy together — patterns interact.

GRACEFUL DEGRADATION AND FALLBACKS

Partial functionality, cached data, default responses and reduced features.



NON-CRITICAL: DEGRADE

- ✓ Recommendations down → hide them
- ✓ Catalog down → acceptable cached data
- ✓ Notifications down → queue for later

CRITICAL: STAY CORRECT

- ! Payment down → PENDING, or reject cleanly

Load shedding

When saturated, reject low-priority work early to protect the critical path.

BUSINESS DECISION Agree fallback behavior with the business, not only in config.

KEY TAKEAWAY Degrade gracefully where you can — but never trade away business correctness.

PART 2: OBSERVABILITY

You can't fix what you can't see.



SO FAR

- ✓ Timeouts bound waiting; retries are selective and keyed
- ✓ Breakers fail fast; bulkheads isolate
- ✓ Fallbacks degrade without faking success

NEXT QUESTION

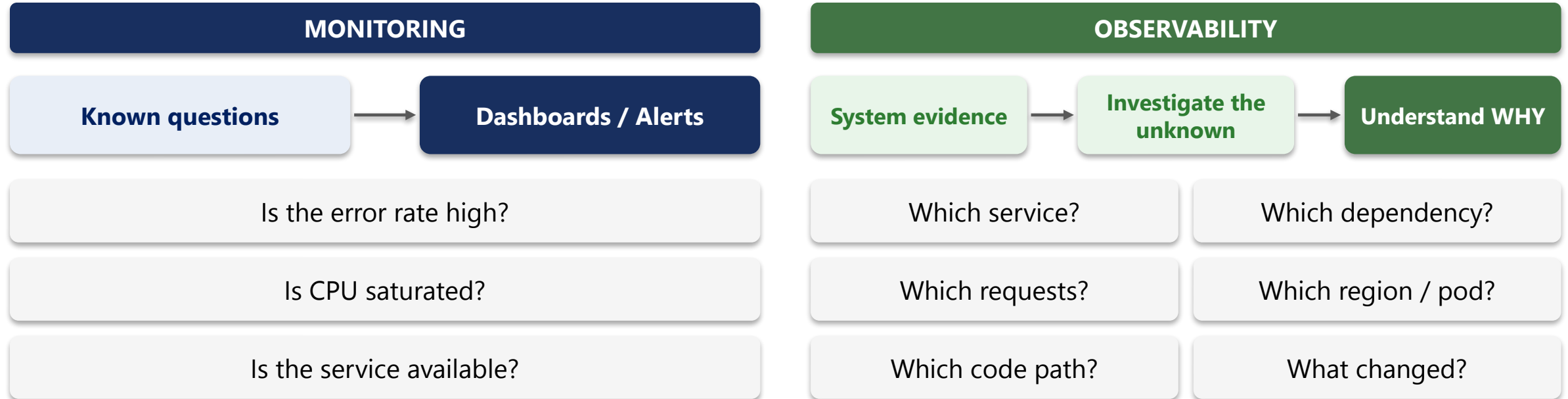
When something still goes wrong in production, how do we see what — and why?

IN THIS PART

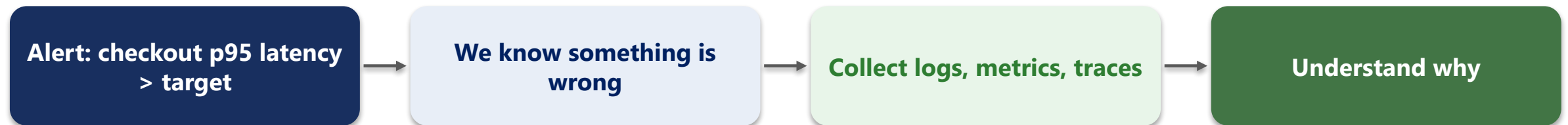
- 1 Observability vs. monitoring
- 2 Logs, metrics and traces
- 3 Structured logs and correlation IDs — in code
- 4 Metrics and health signals — in code
- 5 Distributed tracing
- 6 Exercise: diagnose a slow dependency

OBSERVABILITY VS. MONITORING

Monitoring says something is wrong; observability helps explain why.



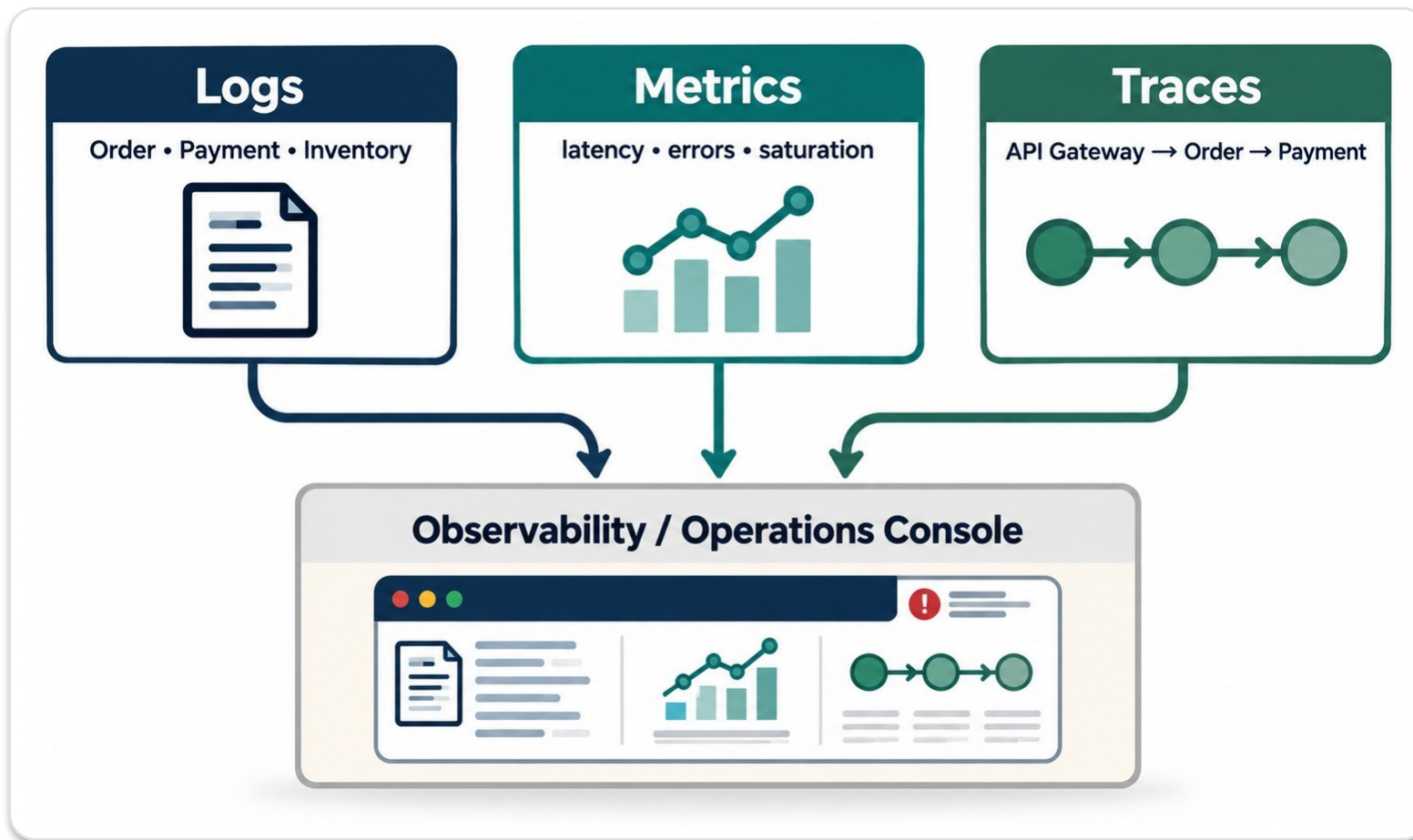
FROM ALERT TO EXPLANATION



KEY TAKEAWAY Monitoring answers known questions; observability lets you investigate new ones.

THE THREE PILLARS OF OBSERVABILITY

Logs, metrics and traces.



USE THEM TOGETHER

- ✓ Metric: checkout latency rose
- ✓ Trace: Payment took 90% of the time
- ✓ Logs: explain the Payment failure

OpenTelemetry

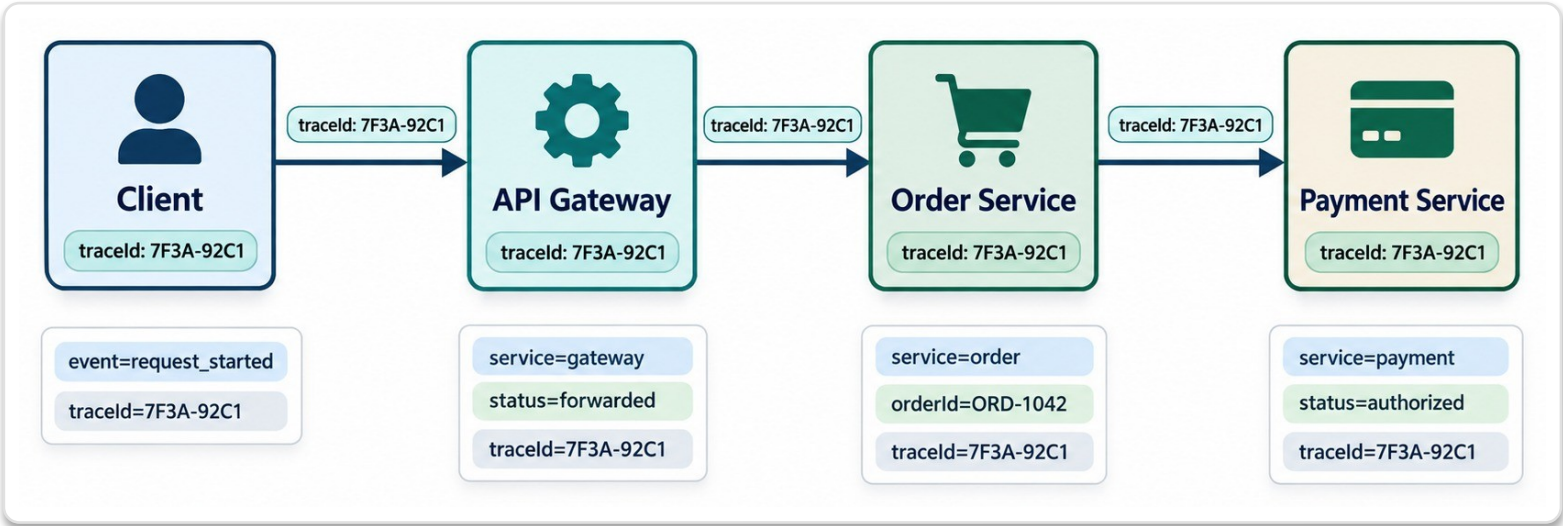
One vendor-neutral standard for all three; Micrometer bridges it in Spring Boot.

RULE No single pillar answers every question.

KEY TAKEAWAY Logs, metrics and traces complement each other — use all three together.

STRUCTURED LOGGING AND CORRELATION IDS

Consistent logs with service, request, trace ID, severity and context.



✖ **Avoid**

ERROR: Something failed

SEARCHABLE FIELDS

service	traceId
orderId	level

NEVER LOG passwords, tokens, card or account numbers.

KEY TAKEAWAY Structured fields and a shared trace ID turn scattered logs into a searchable story.

CORRELATION IDS IN SPRING BOOT

Accept or create an ID, put it in every log line, and pass it on.

✓ CorrelationIdFilter.java

```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Correlation-Id"))
            .orElseGet(() -> UUID.randomUUID().toString());
        MDC.put("correlationId", id);
        res.setHeader("X-Correlation-Id", id);
        try { chain.doFilter(req, res); }
        finally { MDC.remove("correlationId"); }
    }
}
```

Log it and pass it on

```
# application.yml – Spring Boot 3.4+
logging:
  structured:
    format:
      console: ecs      # JSON logs to stdout

// calling the next service
accountApi.get()
    .uri("/accounts/{id}", id)
    .header("X-Correlation-Id", MDC.get("correlationId"))
    .retrieve()
    .body(Account.class);
```

MDC

Every log line on this request thread includes the ID.

Structured

JSON fields that log platforms can search and filter.

Tracing

Micrometer Tracing adds and propagates traceId automatically.

KEY TAKEAWAY One ID, accepted at the edge, written into every log and passed to every downstream call.

METRICS AND HEALTH MONITORING

Latency, error rate, throughput, saturation, JVM and dependency metrics.

TRAFFIC

How much work arrives?

Requests / second

Messages / second

LATENCY

How long does work take?

p50

p95

p99

ERRORS

How often does work fail?

5xx rate

Failed payment %

SATURATION

How close to resource limits?

Thread / DB pool usage

CPU · memory

Queue backlog

ALSO WATCH

CPU

Memory

JVM

GC

Thread pools

Connection
pools

Queue depth

Dependency
latency

DON'T BE FOOLED Low CPU isn't healthy if the DB connection pool is exhausted.

KEY TAKEAWAY Traffic, latency, errors and saturation show how a service is really behaving.

METRICS WITH MICROMETER AND ACTUATOR

Count business outcomes, time dependencies, and expose them for scraping.

✓ PaymentService.java

```
@Service
class PaymentService {
    private final Counter failed;
    private final Timer latency;

    PaymentService(MeterRegistry registry) {
        failed = registry.counter("payments.failed");
        latency = Timer.builder("payments.latency")
            .publishPercentiles(0.95, 0.99)
            .register(registry);
    }

    PaymentResult charge(Order order) {
        return latency.record(() -> doCharge(order));
    }
}
```

application.yml

```
management:
  endpoints:
    web:
      exposure:
        include: health, metrics, prometheus

# with micrometer-registry-prometheus:
# GET /actuator/prometheus
# payments_latency_seconds{quantile="0.95"}
```

Business metrics

Failed payments matter more than CPU.

Percentiles

p95 and p99 show the tail users actually feel.

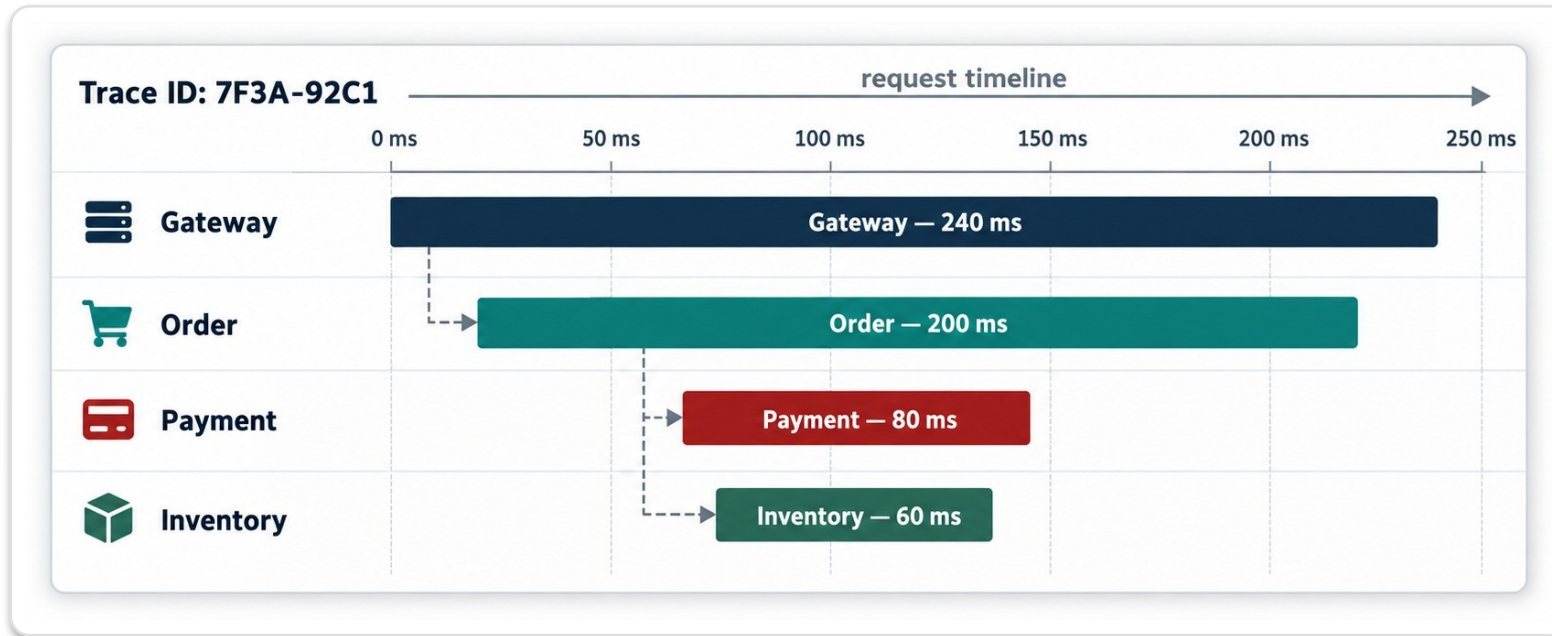
Built in

Resilience4j publishes breaker state and retry counts too.

KEY TAKEAWAY Measure the golden signals and the business outcome — then expose them for dashboards and alerts.

DISTRIBUTED TRACING ACROSS MICROSERVICES

Client → Order → Payment → Inventory — one trace spanning services.



Trace vs. span

A trace is the whole request; a span is one timed operation inside it.

TRACING HELPS WITH

Latency analysis

Failure propagation

Dependency mapping

Cross-service debugging

SPRING BOOT Micrometer Tracing adds and propagates trace IDs automatically.

KEY TAKEAWAY Traces break one request into spans, so the slow service is obvious.

EXERCISE: DIAGNOSE A SLOW DEPENDENCY

Use the signals to find the cause — and choose the fix.

PRACTICE EXERCISE

12 MIN

SCENARIO

Since 10:05, checkout p95 is 4.8 s and 3% of requests fail with 5xx.

Transaction Service calls Fraud Detection during payment. Fraud Detection is slow and starting to time out.

YOUR TASKS

- 1 Which signal do you check first — and what will it show?
- 2 Choose timeout, retry and circuit-breaker settings
- 3 Pick a business-safe fallback
- 4 List the metrics, logs and trace fields you need

EXPECTED OUTCOME

- ✓ Trace: the fraud span dominates the latency
- ✓ Timeout inside the budget; no blind retries on the write
- ✓ A circuit breaker on the fraud call
- ✓ Fallback: PENDING_REVIEW — never auto-approve
- ✓ Fraud p95, timeouts, breaker state; traceld in logs

KEY TAKEAWAY Let the signals locate the problem, then contain it with patterns that keep money movement correct.

PART 3: RELIABILITY TARGETS

Decide what “healthy enough” means — then measure it.



SO FAR

- ✓ Logs, metrics and traces work together
- ✓ One correlation ID follows every request
- ✓ Micrometer and Actuator expose the signals

NEXT QUESTION

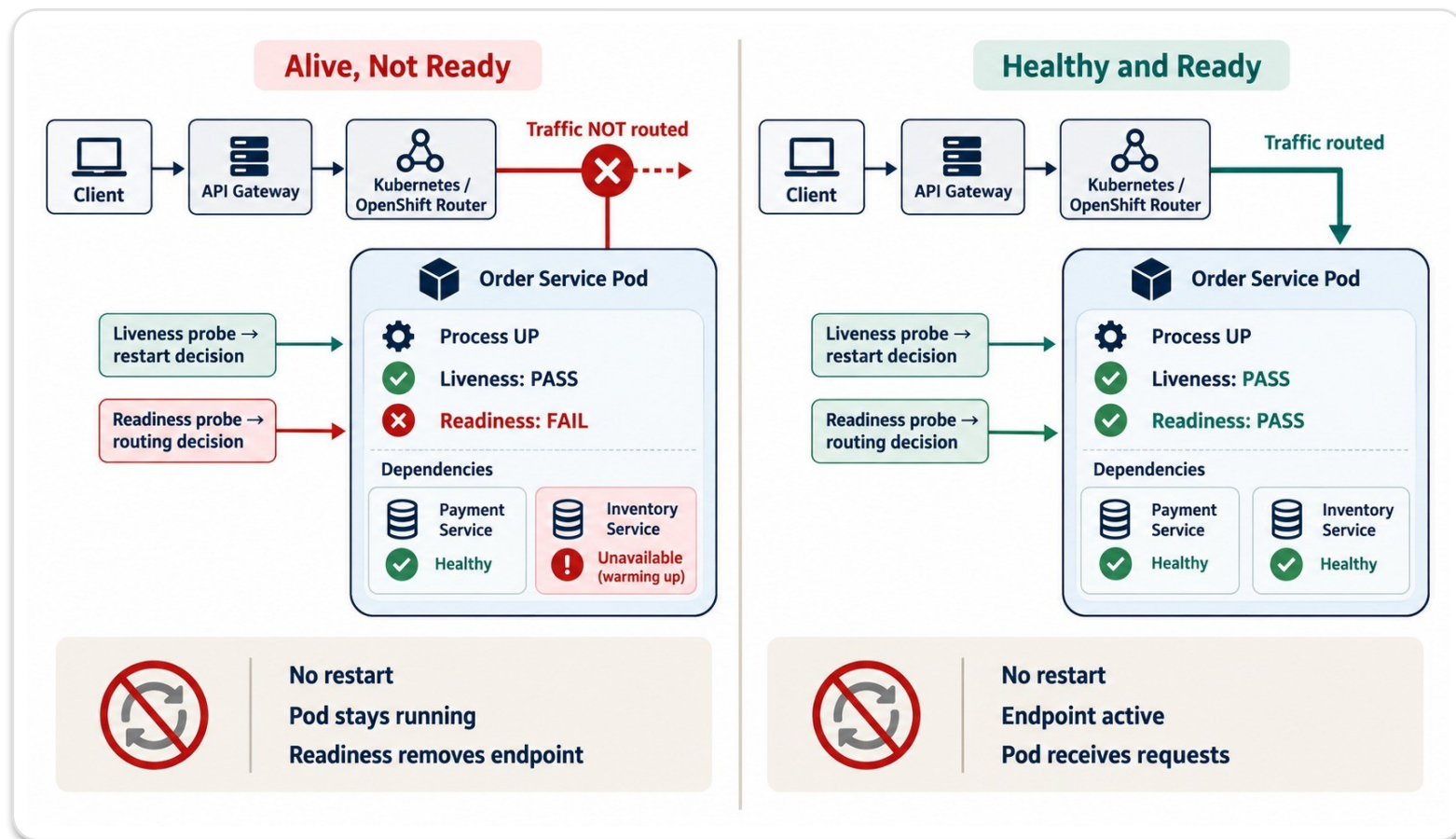
How healthy is healthy enough — and how do we prove it?

IN THIS PART

- 1 Liveness and readiness
- 2 SLIs, SLOs, SLAs and error budgets
- 3 Performance and capacity indicators
- 4 Exercise: define SLOs for banking APIs

HEALTH CHECKS AND READINESS/LIVENESS

Process alive versus service ready to receive traffic.



✓ Spring Boot Actuator

```
management:
  endpoint:
    health:
      probes:
        enabled: true
# /actuator/health/liveness
# /actuator/health/readiness
```

KEEP CHECKS HONEST

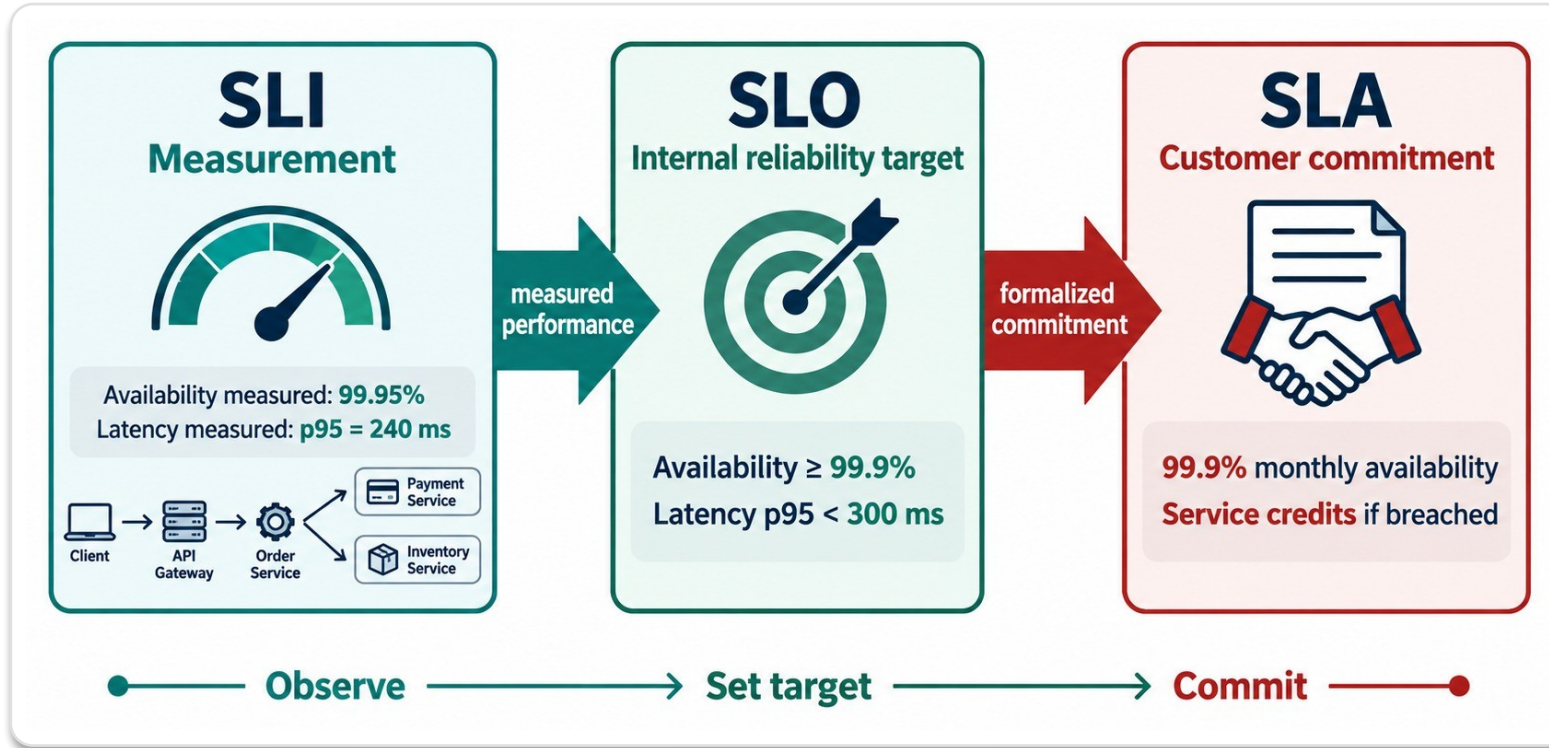
- ✓ Liveness: the app itself only
- ✓ Readiness: may include required dependencies

WARNING A liveness check that calls a dependency causes restart storms.

KEY TAKEAWAY Liveness decides restarts; readiness decides traffic — OpenShift uses both.

SERVICE-LEVEL AGREEMENTS, SLOS AND SLIS

SLA vs. SLO vs. SLI — availability and latency targets.



Error budget

A 99.9% monthly SLO allows about 43 minutes of failure.

USE IT TO DECIDE

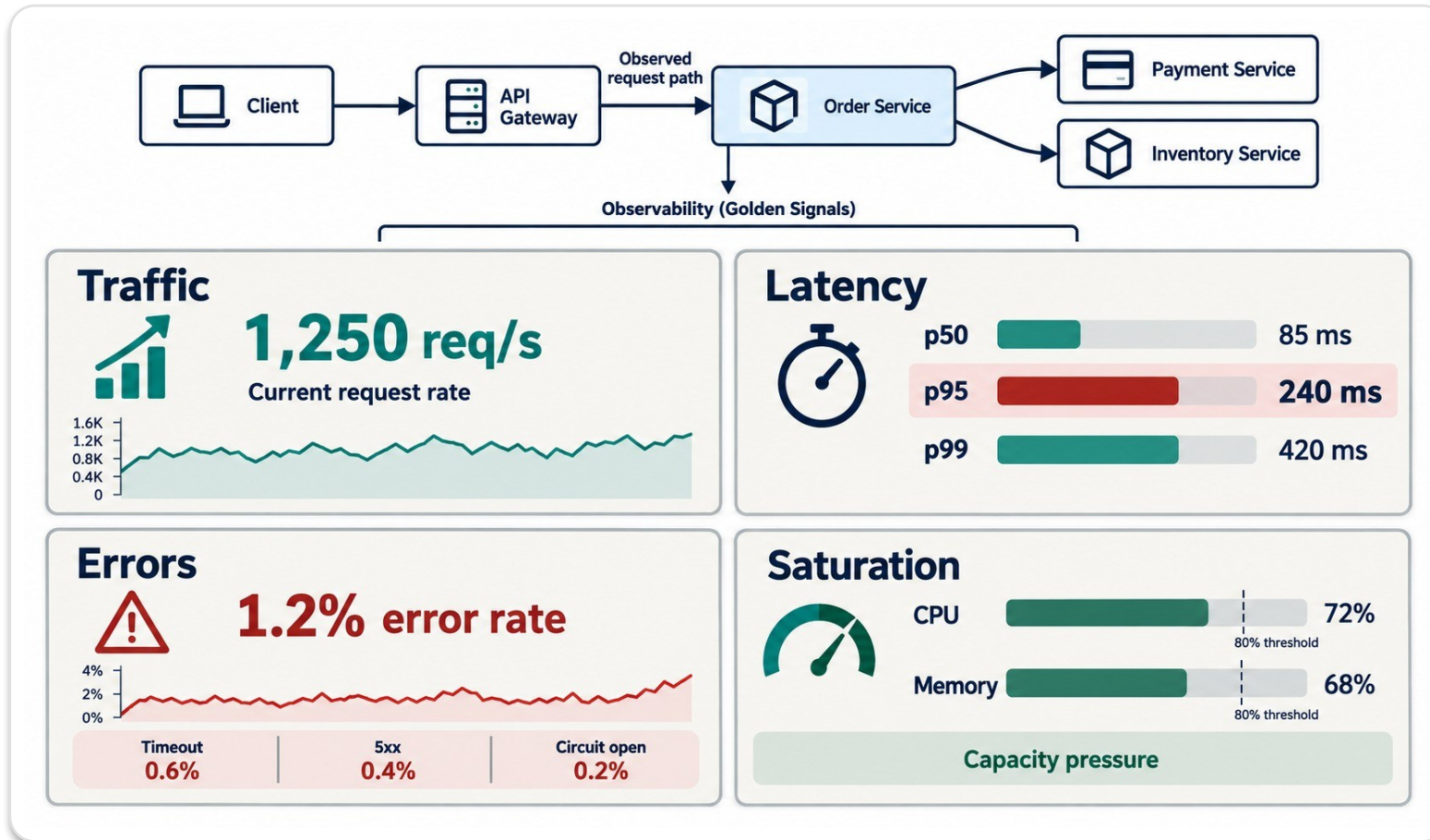
- ✓ Budget left: ship features
- ✓ Budget spent: freeze and fix reliability

NOT EVERY SERVICE Payments deserve stricter targets than promotions.

KEY TAKEAWAY Measure with SLIs, aim with SLOs, and commit with SLAs.

PERFORMANCE AND CAPACITY INDICATORS

p95/p99 latency, throughput, saturation, concurrency and resource use.



Averages hide the tail

An average of 200 ms can hide a p99 of 8 s.

Context matters

1,250 requests per second means little without latency and errors at that load.

LOAD TEST TYPES

Stress

Spike

Soak

Scalability

HEADROOM Plan capacity before saturation, not after.

KEY TAKEAWAY Performance is multidimensional — watch the tail, not just the average.

EXERCISE: DEFINE SLOS FOR BANKING APIS

Turn “reliable enough” into numbers you can measure.

PRACTICE EXERCISE

10 MIN

SCENARIO

Set reliability targets for these APIs:

Account balance API
Transfer API
Fraud check API
Statement download
Rewards lookup

YOUR TASKS

- 1 Pick one SLI per API
- 2 Set an SLO — availability and p95 latency
- 3 Rank them by strictness and explain why

EXPECTED OUTCOME

- ✓ Balance: 99.9%, p95 < 300 ms
- ✓ Transfer: 99.95%, no duplicate transfers
- ✓ Fraud check: p95 < 700 ms
- ✓ Statements: p95 < 3 s
- ✓ Rewards: 99.5% — least critical

KEY TAKEAWAY Strict targets where money moves; relaxed ones where a delay only annoys.

PART 4: CONTINUOUS VALIDATION

A resilience setting nobody tested is only a hope.



SO FAR

- ✓ Liveness restarts; readiness routes traffic
- ✓ SLIs measure, SLOs target, SLAs commit
- ✓ Watch p95, p99, errors and saturation

NEXT QUESTION

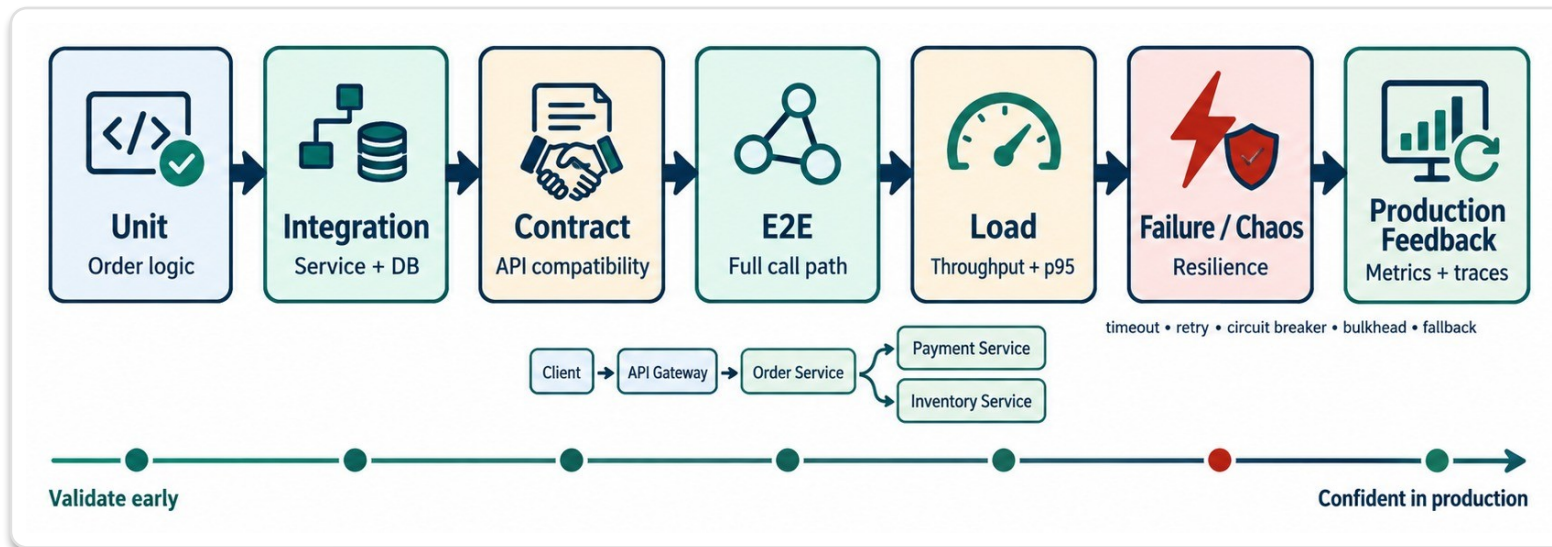
How do we know the resilience we designed actually works — before customers find out?

IN THIS PART

- 1 The continuous testing pipeline
- 2 Testing a timeout fallback — in code
- 3 Exercise: plan a failure experiment
- 4 The complete resilient architecture
- 5 Misconceptions and anti-patterns

CONTINUOUS TESTING AND RESILIENCE VALIDATION

Unit, integration, contract, load, failure, chaos and deployment validation.



SIMULATE

Timeout

503

Slow dependency

DB unavailable

Pod restart

VERIFY

- ✓ The timeout fires
- ✓ Retries stay bounded
- ✓ The circuit opens
- ✓ The fallback is safe
- ✓ The service recovers

QUALITY GATE Fail the build when p95 or error thresholds break.

KEY TAKEAWAY A resilience mechanism never exercised under failure is only an unverified assumption.

TESTING A TIMEOUT FALLBACK

Make the dependency slow in a test — then assert the business rule.

✓ TransactionResilienceTest.java

```
@Test
void slowAccountServiceReturns503() {
    wireMock.stubFor(get(urlPathMatching("/accounts/.*"))
        .willReturn(aResponse()
            .withFixedDelay(5_000)    // beyond the 3 s read timeout
            .withStatus(200)));

    mockMvc.perform(post("/api/v1/transactions")
        .contentType(APPLICATION_JSON)
        .content(validRequestJson()))
        .andExpect(status().isServiceUnavailable())
        .andExpect(jsonPath("$.code")
            .value("ACCOUNT_SERVICE_UNAVAILABLE"));
}
```

...and nothing was published

```
@MockitoBean
KafkaTemplate<String, TransactionSubmitted> kafka;

@Test
void noEventWhenAccountServiceIsDown() {
    // same slow stub, same request as above
    submitTransaction();

    verify(kafka, never())
        .send(eq("transactions.submitted"), any(), any());
}
```

Slow, not down

A delay past the read timeout is the realistic failure.

Assert the rule

503 and no TransactionSubmitted — never a fake success.

Run in CI

Every build proves the fallback still works.

KEY TAKEAWAY Inject the failure in a test and assert the business outcome — every build, not once.

EXERCISE: PLAN A FAILURE EXPERIMENT

Prove a resilience claim safely, before production proves it for you.

PRACTICE EXERCISE

10 MIN

SCENARIO

Your team claims Order Service survives a slow Payment Service. You will test it in staging this afternoon.

YOUR TASKS

- 1 Write the hypothesis you want to prove
- 2 Choose the fault to inject and limit its blast radius
- 3 List what you'll measure — and when you'll stop

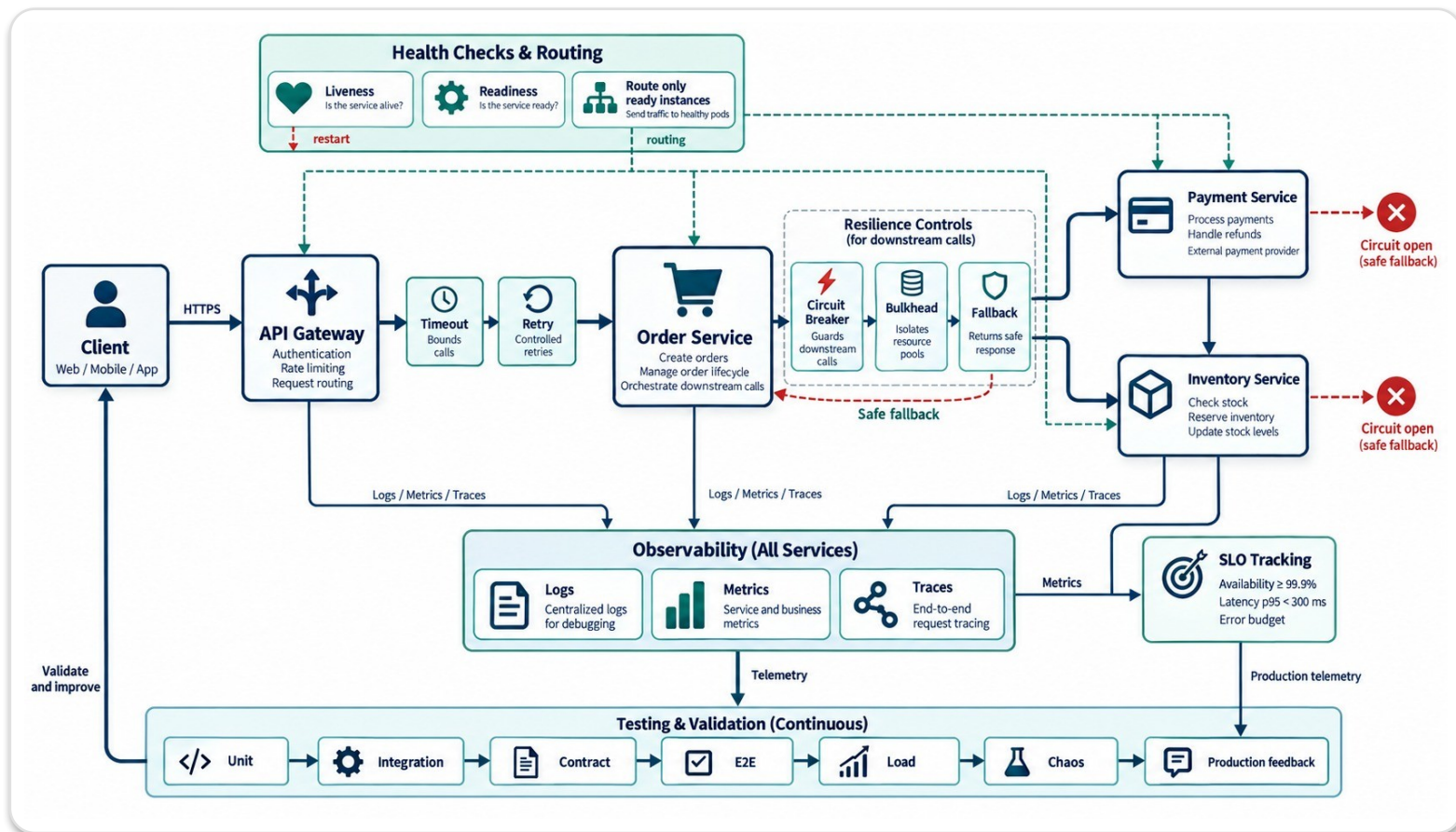
EXPECTED OUTCOME

- ✓ Checkout p95 stays under 2 s; orders go PENDING
- ✓ Add 3 s latency to 10% of Payment calls
- ✓ Watch p95, errors, breaker state, pending orders
- ✓ Abort if the error rate passes 2%
- ✓ Record results, fix, and rerun

KEY TAKEAWAY Chaos engineering is a controlled experiment: hypothesis, small blast radius, clear stop rule.

END-TO-END RESILIENT MICROSERVICES ARCHITECTURE

Resilience patterns, observability, health checks, SLOs and testing feedback.



EVERY PART OF THE MODULE

- ✓ Gateway: timeout, retry, rate limit
- ✓ Order: breaker, bulkhead, fallback
- ✓ Probes route only to ready pods
- ✓ Logs, metrics and traces from every service
- ✓ SLOs and error budgets from telemetry
- ✓ Tests from unit to chaos feed back

REMEMBER Contain, observe, measure — then keep validating.

KEY TAKEAWAY Contain failures, make them observable, measure reliability — and keep testing it.

RESILIENCE MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that turn small failures into outages.

MISCONCEPTION

REALITY

Retries make services more reliable

Blind retries amplify outages

Longer timeouts are safer

They hold threads and hide failures

A circuit breaker fixes the dependency

It only protects the caller

A fallback may return success

Never invent a financial outcome

Average latency tells the story

p95 and p99 are what users feel

ANTI-PATTERNS TO AVOID

- ✗ No timeout on remote calls
- ✗ Retrying non-idempotent writes
- ✗ Retries at every layer
- ✗ Liveness checks that call dependencies
- ✗ Alerting on CPU but not on errors
- ✗ Resilience never tested under failure

REMEMBER Budget latency, then test it — before an incident does.

KEY TAKEAWAY Resilience comes from bounded, deliberate and tested behavior — not from adding more retries.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 Why design microservices for failure?

2 What problem does a timeout solve?

3 When is a retry appropriate?

4 Why can retries be dangerous?

5 What are the three circuit breaker states?

6 What does the bulkhead pattern protect?

7 Which HTTP status signals rate limiting?

8 Why are correlation IDs important?

9 What are the four golden signals?

10 What's the difference between a trace and a span?

MODULE 6 SUMMARY AND WHAT'S NEXT

From fragile call chains to contained, observable and tested services.

YOU CAN NOW

- ✓ Bound every remote call with a timeout inside a budget
- ✓ Retry safely with backoff, jitter and idempotency
- ✓ Protect callers with circuit breakers, bulkheads and fallbacks
- ✓ Diagnose incidents with logs, metrics and traces
- ✓ Set SLOs and prove resilience with failure tests

NEXT: MODULE 7 — DEVOPS AND CI/CD

- 1 How does a code change reach production safely?
- 2 What belongs in a microservices CI/CD pipeline?
- 3 How do we deploy without downtime — and roll back?

BRING ALONG Your Lab 3 Transaction Service with its timeouts and circuit breaker.

KEY TAKEAWAY Module 7 automates getting these services to production safely.